```
  d3.selectAll("circle.node")
    .attr("cx", function(d) {return d.x})
    .attr("cy", function(d) {return d.y});
};
```

Draws nodes as SVG

The rendering of the network is similar in appearance, as you can see in figure 11.10, but the performance improves dramatically. Using canvas, I can draw 10,000 link networks with performance high enough to have animation and interactivity. The canvas drawing code can be a bit cumbersome (it's like the old LOGO drawing code), but the performance makes it more than worth it.

We could use the same method as with the earlier maps to use canvas during animated periods and SVG when the network is fixed. But we'll move on and look at another method for dealing with large amounts of data: quadtrees.

11.3 Optimizing xy data selection with quadtrees

When you're working with a large dataset, one issue is optimizing search and selection of elements in a region. Let's say you're working with a set of data with xy coordinates (anything that's laid out on a plane or screen). You've seen enough examples in this book to know that this may be a scatterplot, points on a map, or any of a number of different graphical representations of data. When you have data like this, you often want to know what datapoints fall in a particular selected region. This is referred to as *spatial search* (and notice that "spatial" in this case doesn't refer to geographic, but rather space in a more generic sense). The quadtree functionality is a spatial version of `d3.nest`, which we used in chapter 5 and chapter 8 and will use again in chapter 12 (available online only) to create hierarchical data. Following the

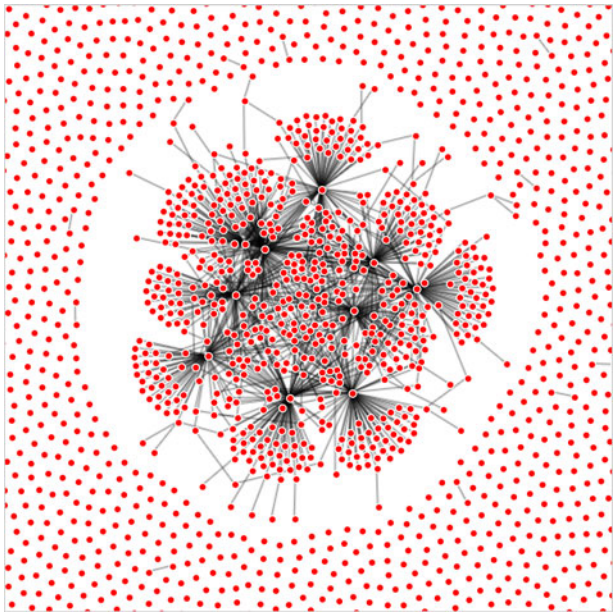


Figure 11.10 A large network drawn with SVG nodes and canvas links